

# Trabalho Final INF1022 2026.1

Profs. Vitor Pinheiro e Edward Hermann

8 de junho de 2026

## 1 Enunciado

O trabalho pode ser feito em dupla ou de forma individual. Neste trabalho deve ser desenvolvido um analisador sintático para a linguagem ObsAct. O analisador sintático deve ser capaz de compilar programas escritos utilizando a linguagem ObsAct para uma outra linguagem a sua escolha (como ilustrado na Figura 1). Ou seja, o analisador sintático recebe como entrada um programa na linguagem ObsAct e produz como saída um outro programa escrito em uma outra linguagem. A linguagem do programa de saída pode ser escolhida por você, ela pode ser qualquer linguagem a sua escolha. Por exemplo, mas não restrita a essas: C, C++, Java, Lua, Python ou Assembly.

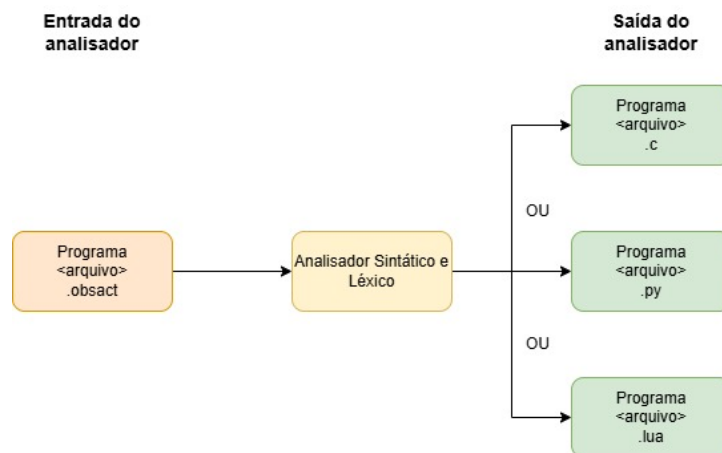


Figura 1: Analisador sintático para a linguagem ObsAct.

Para a implementação do analisador sintático, deve-se usar um gerador de analisador sintático que implemente o método LaLR(1) ou outro ascendente, por exemplo, Yacc/Lex, Bison/Flex (usa LaLR(1)), JavaCC (que usa o descendente LL(1)), etc.

## 1.1 Desenvolvimento

A sintaxe da linguagem *ObsAct* é dada pela gramática abaixo:

|                   |               |                                                         |
|-------------------|---------------|---------------------------------------------------------|
| <i>PROGRAM</i>    | $\rightarrow$ | <i>DEVICES CMDS</i>                                     |
| <i>DEVICES</i>    | $\rightarrow$ | <i>DEVICE DEVICES</i>   <i>DEVICE</i>                   |
| <i>DEVICE</i>     | $\rightarrow$ | <i>dispositivo</i> : { <i>namedevice</i> }              |
| <i>DEVICE</i>     | $\rightarrow$ | <i>dispositivo</i> : { <i>namedevice, observation</i> } |
| <i>CMDS</i>       | $\rightarrow$ | <i>CMD. CMDS</i>   <i>CMD.</i>                          |
| <i>CMD</i>        | $\rightarrow$ | <i>ATTRIB</i>   <i>OBSACT</i>   <i>ACT</i>              |
| <i>ATTRIB</i>     | $\rightarrow$ | <i>set observation = VAR</i>                            |
| <i>ATTRIB</i>     | $\rightarrow$ | <i>set observation = ACTEXECUTE</i>                     |
| <i>OBSACT</i>     | $\rightarrow$ | <i>se OBS entao CMDS</i>                                |
| <i>OBSACT</i>     | $\rightarrow$ | <i>se OBS entao CMDS senao CMDS</i>                     |
| <i>OBS</i>        | $\rightarrow$ | <i>observation oplogic VAR</i>                          |
| <i>OBS</i>        | $\rightarrow$ | <i>observation oplogic VAR &amp;&amp; OBS</i>           |
| <i>VAR</i>        | $\rightarrow$ | <i>num</i>   <i>bool</i>                                |
| <i>ACT</i>        | $\rightarrow$ | <i>ACTEXECUTE</i>   <i>ACTALERT</i>                     |
| <i>ACTEXECUTE</i> | $\rightarrow$ | <i>ACTION namedevice</i>                                |
| <i>ACTALERT</i>   | $\rightarrow$ | <i>enviar alerta (msg) namedevice</i>                   |
| <i>ACTALERT</i>   | $\rightarrow$ | <i>enviar alerta (msg, observation) namedevice</i>      |
| <i>ACTION</i>     | $\rightarrow$ | <i>ligar</i>   <i>desligar</i>   <i>verificar</i>       |

- Todas as variáveis numéricas são do tipo inteiro e não negativo.
- Todos os terminais (por exemplo: *num*, *namedevice* e *msg*) não podem ser vazios.
- *num* representa um número.
- *bool* representa TRUE ou FALSE.
- *observation* é uma string que representa o nome do sensor a ser observado.
- O terminal *namedevice* só pode conter letras.
- O terminal && é o operador lógico somente utilizado entre não terminais *OBS*. Significa que duas *OBS* precisam ser verdadeiras ao mesmo tempo. Por exemplo: se movimento == True && luz < 100 entao ligar lampada.
- Perceba que o terminal "." sinaliza o fim dos comandos *ATTRIB* e *OBSACT*.
- *oplogic*: é um terminal que representa um operador lógico. Eles podem ser: >, <, >=, <=, == ou !=

A regra:

*ACT*  $\rightarrow$  *enviar alerta (msg, observation) namedevice*

diz para enviar um alerta (msg, observation) para o device com nome *namedevice*. A mensagem de alerta contém duas strings que devem ser concatenadas: msg e

observation. A concatenação resultante deve ser `msg + (espaço em branco) + observation`. Por exemplo, a linha de código:

```
1 enviar alerta ("Temperatura medida está em", temperatura)
   Termometro .
```

vai concatenar a string "Temperatura medida está em" com a variável *temperatura* e vai resultar na string "Temperatura medida está em 37", sendo que 37 seria o valor da variável *temperatura*.

A gramática acima não está completa, neste trabalho você vai precisar definir algumas regras a mais para incluir algumas funcionalidades. **Mais importante que isso, fique a vontade para alterar a gramática caso ache necessário para poder gerar a linguagem que está sendo pedida.** Por exemplo, você pode retirar recursão a esquerda, fatorar ou até alterar as regras acima. Desde que a linguagem final gerada seja a mesma.

Além disso, a gramática descrita acima deve ser complementada ou corrigida para que a linguagem ObsAct seja capaz de executar comandos do tipo:

- Envio broadcast: será uma forma que o programador vai ter para enviar um alerta para um conjunto de dispositivos. Por exemplo:

```
1 enviar alerta ("mensagem") para todos: namedevice,
   namedevice .
```

Neste caso pode-se colocar quantos *namedevice* desejar e o alerta deve ser enviado para todos eles.

## 1.2 Exemplos de código na linguagem Obs

Alguns exemplos de programas em ObsAct são:

```
1 dispositivo: {Termometro, temperatura}
2 dispositivo: {ventilador, potencia}
3 set temperatura = 40.
4 set potencia = 90.
5 se temperatura > 30 entao ligar ventilador.
```

```
1 dispositivo: {Termometro, temperatura}
2 dispositivo: {ventilador, potencia}
3 set temperatura = 40.
4
5 se temperatura > 30 entao
6     set estado_ventilador = verificar(ventilador).
7     se estado_ventilador == 0 entao ligar ventilador.
8     set potencia = 90.
9 .
```

```

1 dispositivo {monitor}
2 dispositivo: {celular}
3 dispositivo: {Termometro, temperatura}
4 se temperatura > 30 entao .
5 enviar alerta ("Temperatura em ", temperatura) para todos:
    monitor, celular .

```

```

1 dispositivo: {celular, movimento}
2 dispositivo: {higrometro, umidade}
3 dispositivo: {lampada, potencia}
4 dispositivo: {Monitor}
5 set potencia = 100 .
6 se umidade < 40 entao enviar alerta "Ar seco detectado"
    Monitor .
7 se movimento == True entao ligar lampada senao desligar
    lampada .

```

```

1 dispositivo: {celular, movimento}
2 dispositivo: {higrometro, umidade}
3 dispositivo: {lampada, potenciaLampada}
4 dispositivo: {umidificador, potenciaUmidificador}
5 dispositivo: {Monitor}
6 set {lampada, potenciaLampada} = 100 .
7 se umidade < 40 entao
8     enviar alerta "Ar seco detectado" Monitor .
9     se verificar(umidificador) == 0 entao
10         ligar umidificador .
11         set potenciaUmidificador = 100 .
12
13 se movimento == True entao ligar lampada senao desligar
    lampada .

```

```

1 dispositivo: {umidade}
2 dispositivo {Monitor}
3 se umidade < 40 entao enviar alerta "Ar seco detectado"
    Monitor .

```

```

1 dispositivo: {lampada, potencia}
2 set potencia = 100
3 ligar lampada

```

```

1 dispositivo: {ventilador}
2 desligar ventilador

```

```

1 dispositivo: {Celular}
2 enviar alerta ("Hora de acordar!") Celular

```

```

1 dispositivo: {Termometro, temperatura}
2 enviar alerta ("Temperatura esta em", temperatura)
   Termometro

```

OBS: Crie seus exemplos de códigos também para testar seu transpilador.

### 1.3 Sobre os dispositivos

Veja que no inicio de todo programa ObsAct o programador precisa primeiro declarar todos os dispositivos que serão utilizados ao longo do programa. Como por exemplo:

```

1 dispositivo: {Termometro, temperatura}
2 dispositivo: {Ventilador, potencia}
3 dispositivo: {Celular}
4 dispositivo: {Monitor}

```

Estas regras estão em algum dos seguintes formatos:

$$\begin{aligned}
 DEVICE &\longrightarrow \text{dispositivo} : \{namedevice\} \\
 DEVICE &\longrightarrow \text{dispositivo} : \{namedevice, observation\}
 \end{aligned}$$

Todo programa gerado a partir de um programa escrito em ObsAct vai conter 4 funções: uma para ligar o dispositivo, outra para desligar o dispositivo, outras duas para enviar um alerta para o dispositivo. Estas funções devem ser implementadas como a seguir:

```

1 function ligar(namedevice)
2 {
3     print(namedevice+" ligado!")
4     return 1
5 }
6
7 function desligar(namedevice)
8 {
9     print(namedevice+" desligado!")
10    return 0
11 }
12
13 function verificar(namedevice)
14 {
15     print(namedevice+" est  ligado.")
16     return 1
17     // OU (depende do estado que ele est )
18     print(namedevice+" est  desligado.")
19     return 0
20 }
21
22 function alerta(namedevice, msg)
23 {

```

```

24     print(namedevice+" recebeu o alerta:\n")
25     print(msg)
26 }
27
28 function alerta(namedevice, msg, var)
29 {
30     print(namedevice+" recebeu o alerta:\n")
31     print(msg + " " + var)
32 }

```

Veja que estas 5 funções definidas acima estão em um pseudocódigo. *namedevice* é a variável que identifica o dispositivo.

Elas devem ser implementadas na linguagem de programação escolhida pelo aluno. Por exemplo, caso o aluno queria traduzir a linguagem ObsAct para Python, as funções acima devem ser geradas em Python.

Desta maneira, toda vez que alguma ação de *ligar*, *desligar*, *verificar* ou *alerta* for executada para algum *namedevice*, são estas funções que devem ser chamadas passando como parametro o *namedevice*.

Para facilitar o trabalho, o aluno pode escolher também já deixar estas 4 funções implementadas no código. Por exemplo, se o aluno for gerar código em Python, ele já pode ter um arquivo .py que vai conter estas 4 funções (ligar, desligar, verificar e as duas de alerta).

As variáveis de *msg* e *namedevice* podem conter no máximo 100 caracteres.

## 1.4 Suposições

Todo valor de *observation* que não for definido pelo programador através da linha de código:

```

1 set temperatura = 40 .

```

deve ser definido para zero.

Perceba que a regra da gramática responsável por definir os valores dos sensores é:

$$ATTRIB \longrightarrow set \ observation = VAR.$$

## 1.5 Observação

Você tem a permissão de expandir a gramática para permitir comandos adicionais ou demais operações que julgar interessantes e/ou necessárias.

Deixe explícitas as alterações realizadas na gramática descrita neste documento (documente no relatório do trabalho). Tanto as alterações obrigatórias (solicitadas neste trabalho) quanto alterações que você julgar interessantes.

## 2 Entrega

Você deve entregar um arquivo contendo seu relatório, no qual estarão descritos seu trabalho - o que foi implementado, como foi implementado, o que funciona, o

que não funciona, quais os testes utilizados etc. - e quaisquer outras informações que você considere relevantes, como a maneira de executá-lo.

No seu relatório final deve estar contido a gramática final utilizada no trabalho. A gramática deve estar descrita de maneira similar ao que a gramática inicial foi descrita neste documento. Deixe claro quais regras foram adicionadas na gramática descrita neste relatório para que seja possível permitir todas as funcionalidades solicitadas. Caso tenha implementado alguma funcionalidade adicional, ou seja, uma que não foi solicitada, por favor deixe explícito.

Além disso, entregue um .zip com os casos de teste utilizados.

Por fim, entregue o código do seu analisador bem como quaisquer outros arquivos/módulos auxiliares que tenha criado para a execução do trabalho.

Indique nome e matrícula de **ambos os componentes da dupla**, se for o caso.

Caso tenha arguições, a ordem das arguições será definida com base na ordem das entregas no EAD.

## Dica

- Comece pequeno: garanta que seu analisador funcione para um fragmento da linguagem, depois vá incrementando.

